

INFORME DE MÉTODOS Y ESTÁNDARES DE CALIDAD DEL SOFTWARE

Marco de calidad adoptado y plan de pruebas de FactuGest, plataforma web de facturación electrónica para la emisión de documentos por cuenta de terceros

Presentado por:

BRANDON ARLEY RESTREPO GÉLVEZ

JOHAN SEBASTIÁN ACOSTA SÁNCHEZ

WILMER JESÚS CONTRERAS RANGEL

Ficha 3115426

Tecnólogo en Análisis y Desarrollo de Software

Evidencia AA1-EV01

Controlar la calidad del servicio de software de acuerdo con los estándares técnicos

Instructora: HEIDY LIZBETH ADARME

SERVICIO NACIONAL DE APRENDIZAJE (SENA)

Centro de la Industria, la Empresa y los Servicios (CIES)

Tecnólogo en Análisis y Desarrollo de Software

CÚCUTA, NORTE DE SANTANDER

2026

Tabla de contenido

1. Introducción	5
2. Marco de calidad del software	7
2.1 Calidad y calidad de software	7
2.2 Modelo y estándar	8
2.3 Clasificación por producto y por proceso	8
2.4 Modelos de calidad a nivel de producto	9
2.5 Modelo de calidad a nivel de proceso	11
2.6 Estándares de calidad a nivel de producto	12
2.7 Estándares de calidad a nivel de proceso	14
3. El referente de calidad adoptado	16
3.1 Por qué la norma ISO/IEC 25010	16
3.2 Los requisitos del sistema contra el modelo de calidad	17
3.3 Donde el catálogo propio y el estándar no coinciden	19
3.4 El referente de proceso, CMMI en su nivel 2	20
3.5 Los referentes que no se adoptaron	22
4. Plan de pruebas	24
4.1 Objetivo	24
4.2 Alcance	24
4.3 Estrategia y niveles de prueba	25

4.4 Entorno y datos de prueba.....	27
4.5 Criterios de entrada y de salida	28
4.6 Gestión de los defectos	28
4.7 Riesgos del plan	29
4.8 Roles y responsabilidades	30
5. Pruebas ejecutadas	32
5.1 Pruebas unitarias	32
5.2 Casos de prueba y sus resultados	33
5.3 Pruebas de integración sobre la interfaz	36
5.4 Pruebas funcionales	37
5.5 Pruebas de seguridad.....	39
5.6 Pruebas de usabilidad.....	40
6. Pruebas planificadas.....	41
7. Resultados	43
7.1 Defectos encontrados y corregidos	43
7.2 Lo que los defectos tienen en común	44
7.3 Cobertura frente al modelo de calidad adoptado	45
8. Conclusiones y plan de mejora	46
9. Referencias.....	48
10. Anexos	51

Anexo A. Salida de la corrida de pruebas	52
Anexo B. Listado de las pruebas superadas.....	53

1. Introducción

El presente informe corresponde a la evidencia AA1-EV01 de la competencia «Controlar la calidad del servicio de software de acuerdo con los estándares técnicos», y recoge el análisis de los marcos de calidad aplicables a FactuGest, el estándar adoptado para el proyecto y el plan de pruebas con el que se validó su funcionamiento. No es un estudio general sobre calidad de software, sino la aplicación de esos marcos a un sistema concreto, construido, en funcionamiento y verificable, de modo que cada afirmación que aquí se hace sobre el software puede comprobarse en el repositorio del proyecto.

FactuGest es una plataforma web de facturación electrónica que opera como proveedor tecnológico para micro, pequeñas y medianas empresas de Colombia. A diferencia de un facturador convencional, no reemplaza el software con el que trabaja la empresa, sino que emite por cuenta de ella a través de una interfaz de integración que su propio sistema consume, de manera que el negocio cumple con la obligación tributaria sin cambiar la herramienta que ya usa. La plataforma expide facturas de venta, notas crédito y notas débito, calcula las bases gravables, los descuentos y el prorrateo de impuestos, reserva el consecutivo autorizado en la resolución de facturación, genera el código único de facturación electrónica, la representación gráfica en formato PDF y el archivo XML bajo el estándar UBL 2.1, y hace llegar el documento al comprador.

Esa condición de proveedor es la que explica por qué la calidad, en este proyecto, no admite el tratamiento habitual. Un defecto en un sistema de gestión corriente afecta a quien lo usa, mientras que aquí un defecto se reproduce en todas las empresas integradas y en todos los documentos que emiten. Además, la factura electrónica es un documento con efectos legales y su contenido está reglado por la autoridad tributaria (DIAN, 2020), así que un error de cálculo, de

numeración o de identificación del emisor no produce una molestia para el usuario sino un documento rechazado o, peor, aceptado diciendo algo que no corresponde. La calidad aquí es una condición para operar y no un atributo deseable del producto.

El informe se organiza en cuatro partes. La primera revisa los modelos y estándares de calidad vigentes, distinguiendo los que miran el producto de los que miran el proceso. La segunda declara cuáles se adoptaron para FactuGest, con el motivo por el que se descartaron los demás, y cruza el estándar elegido contra los requisitos no funcionales del sistema. La tercera presenta el plan de pruebas, con su alcance, su entorno, sus criterios y sus riesgos. La cuarta detalla las pruebas ejecutadas, con sus casos y resultados, las que quedan planificadas y los defectos que el proceso permitió encontrar y corregir.

2. Marco de calidad del software

Antes de elegir un referente conviene fijar cuatro términos que el lenguaje corriente usa como equivalentes y que en este campo no lo son, pues de la diferencia entre modelo y estándar depende buena parte de lo que se decide en el capítulo siguiente.

2.1 Calidad y calidad de software

La palabra calidad es ambigua y depende de quién observa, razón por la cual el primer trabajo de cualquier marco es volverla medible. Vega et al. (2008) lo plantean en una frase que sirve de punto de partida, «pues si la calidad no puede ser definida, no puede ser medida; y donde la calidad no puede ser medida, entonces no puede ser controlada» (p. 145). El diccionario recoge el uso común del término como el conjunto de propiedades inherentes que permiten juzgar el valor de algo (Real Academia Española, 2023), lo que confirma el problema más que resolverlo, ya que juzgar el valor de un programa exige decir antes contra qué se lo juzga.

Aplicada al software, la definición más citada es la de Pressman (2010), para quien la calidad es «la concordancia con los requerimientos funcionales y de rendimiento explícitamente establecidos, con los estándares de desarrollo explícitamente documentados y con las características implícitas que se espera de todo software desarrollado profesionalmente» (p. 294). La definición tiene tres partes y las tres cuentan, porque un producto puede cumplir lo que se le pidió, hacerlo sin seguir ninguna práctica reconocida y fallar en lo que nadie escribió porque se daba por supuesto.

La norma ISO/IEC 9126 la define como la totalidad de características de un producto de software que le dan la capacidad de satisfacer necesidades explícitas o implícitas del cliente (ISO/IEC, 2001), y el IEEE, como el grado con el que un sistema, componente o proceso cumple los requisitos especificados y las expectativas del usuario (IEEE, 1990). Las tres definiciones

coinciden en algo que conviene retener, y es que la calidad no se mide contra un ideal abstracto sino contra requisitos declarados, de modo que un proyecto sin requisitos escritos no tiene forma de afirmar que su producto tiene calidad. Conviene además la advertencia que recoge Gómez (2009) al citar a Mike Bria, según la cual la calidad se refiere a la ausencia de defectos antes que a la presencia de valor, pues son dos cosas que suelen confundirse al evaluar un producto.

2.2 Modelo y estándar

Un modelo es un esquema teórico que explica una realidad compleja para facilitar su comprensión y su estudio (Real Academia Española, 2023). En calidad de software, un modelo integra buenas prácticas, señala los temas en los que una organización debe poner énfasis y permite medir los avances (Scalone, 2006). Un estándar, en cambio, es un conjunto de guías o patrones adoptados por consenso y validados por una o varias entidades acreditadas, cuya función es que todos los procesos se realicen de la misma forma.

La diferencia práctica es de origen y de exigibilidad. El modelo propone y ordena, mientras que el estándar norma y permite comparar, certificar y exigir contractualmente. Para el proyecto esto tiene una consecuencia inmediata, porque un modelo se puede adoptar parcialmente y adaptar sin faltar a nada, mientras que un estándar se cumple o no se cumple, y afirmar que se cumple obliga a poder mostrar cómo.

2.3 Clasificación por producto y por proceso

Modelos y estándares se dividen además según lo que observan. Los de producto miran el software terminado y sus atributos, mientras que los de proceso miran la forma de trabajar de la organización que lo construye, bajo el supuesto de que un proceso maduro produce productos consistentes. Las dos miradas son complementarias y ninguna sustituye a la otra, ya que un

proceso impecable no garantiza que el producto sirva y un producto correcto puede haber salido de un proceso irrepetible.

Tabla 1

Clasificación de los modelos y estándares de calidad del software

Ámbito	Modelos	Estándares
Producto	McCall (1977), Boehm (1978), FURPS, Dromey	ISO/IEC 9126, ISO/IEC 25000 SQuaRE, ISO/IEC 25010
Proceso	CMM, CMMI, Bootstrap, PSP, TSP, Six Sigma para software	ISO/IEC 15504 SPICE, ISO 9001, ISO/IEC 12207, TickIT

Nota. Adaptado de Scalone, 2006 y del material de formación (SENA, s.f.). Se destacan en el texto los que se estudian en este informe.

El informe estudia los cinco que el material de formación desarrolla, que son los modelos de McCall y Boehm y el modelo de proceso CMMI, junto con los estándares ISO/IEC 9126, ISO/IEC 25000, ISO/IEC 25010, ISO/IEC 15504 e ISO 9001. Sobre esa revisión se decide en el capítulo 3.

2.4 Modelos de calidad a nivel de producto

2.4.1 Modelo de McCall.

Desarrollado en 1977 por Jim McCall para la fuerza aérea de los Estados Unidos, busca reducir la distancia entre usuarios y desarrolladores tomando en cuenta factores de calidad en los que estén presentes las necesidades de ambos (Estayno et al., 2009). Su propuesta es especificar los requisitos de calidad al comenzar el proyecto y en cada etapa del ciclo de vida, fijando para cada factor un valor deseable que al final se comprueba, lo que lo convierte en el primer modelo que trata la calidad como algo que se planea y no como algo que se revisa al terminar.

El modelo organiza once factores en tres ejes o puntos de vista desde los cuales el usuario puede observar el producto, y cada factor se descompone en criterios que se evalúan mediante métricas con valores máximos y mínimos.

Tabla 2

Los once factores de calidad del modelo de McCall

Eje	Qué observa	Factores
Operación del producto	Cómo se comporta mientras se usa	Corrección, fiabilidad, eficiencia, integridad y facilidad de uso
Revisión del producto	Qué tan fácil es corregirlo y adaptarlo	Facilidad de mantenimiento, flexibilidad y facilidad de prueba
Transición del producto	Qué tan bien se mueve a otro entorno	Portabilidad, reusabilidad e interoperabilidad

Nota. Adaptado de Pressman, 2010 y Estayno et al., 2009.

Su límite es conocido y conviene anotarlo, porque varios de sus factores no tienen métrica directa y terminan evaluándose con listas de comprobación subjetivas. Aun así, la partición en tres ejes sigue siendo útil como forma de ordenar la conversación sobre qué se le va a exigir a un producto.

2.4.2 Modelo de Boehm.

Presentado por Barry Boehm en 1978, es el segundo más reconocido y se organiza en tres niveles que van de lo general a lo medible. En el nivel alto están los requerimientos generales de uso, en el intermedio los factores de calidad propiamente dichos y en el primitivo las características asociadas de forma directa a una o dos métricas.

Tabla 3

Los tres niveles del modelo de Boehm

Nivel	Contenido
Alto	Utilidad per se, o qué tan usable, confiable y eficiente es el producto en sí mismo. Mantenibilidad, o qué tan fácil es entenderlo, modificarlo y volverlo a probar. Utilidad general, o si puede seguir usándose al cambiar el entorno
Intermedio	Portabilidad, confiabilidad, eficiencia, usabilidad, testeabilidad, facilidad de entendimiento y modificabilidad
Primitivo	Características medibles de forma directa, que son las que alimentan las métricas del nivel superior

Nota. Adaptado del material de formación (SENA, s.f.).

Frente a McCall, el aporte de Boehm es la trazabilidad entre niveles, ya que cada característica general se sostiene en otras que sí se pueden medir. Es la idea que recogen después los estándares ISO, donde cada característica se descompone en subcaracterísticas y estas en métricas.

2.5 Modelo de calidad a nivel de proceso

A nivel de proceso existen varios referentes, entre ellos CMM y su sucesor CMMI, Bootstrap, TickIT, el proceso personal de software, el proceso de software en equipo y Six Sigma para software. El material de formación desarrolla CMMI por su incidencia en el mercado, y es el que se revisa aquí.

2.5.1 CMMI.

El modelo de integración de madurez de capacidades es un enfoque de mejora de procesos publicado en el año 2000 por el Instituto de Ingeniería de Software de la Universidad Carnegie Mellon (CMMI Product Team, 2010). Permite a una organización determinar la madurez de sus procesos actuales, identificar los elementos críticos para el aseguramiento de la calidad y reconocer las prácticas clave que necesita incorporar para mejorar. A diferencia de CMM, que estaba pensado sobre el modelo en cascada, CMMI se formuló para los modelos de desarrollo iterativos que se usan hoy, y esa es la razón por la que sigue siendo aplicable a un proyecto que trabaja por iteraciones.

Tabla 4

Los cinco niveles de madurez de CMMI

Nivel	Nombre	Qué caracteriza a la organización
1	Inicial	Los procesos son caóticos y el resultado depende del esfuerzo de las personas
2	Gestionado	Hay conciencia de la dirección y los procesos se planifican y se controlan por proyecto
3	Definido	Los procesos están caracterizados, comprendidos y son los mismos en toda la organización

Nivel	Nombre	Qué caracteriza a la organización
4	Gestión cuantitativa	Se establecen objetivos medibles de rendimiento y calidad del proceso
5	En optimización	La mejora es continua y se apoya en la comprensión cuantitativa de los resultados

Nota. Adaptado de CMMI Product Team, 2010 y del material de formación (SENA, s.f.).

Para un equipo de formación el valor de CMMI no está en la certificación, que supone una evaluación formal fuera del alcance de este proyecto, sino en el orden que propone. El nivel 2 describe prácticas que un equipo pequeño sí puede sostener, como la gestión de requisitos, la planificación y el seguimiento del trabajo, la gestión de configuración y el aseguramiento de la calidad del producto y del proceso, y es contra ese nivel que el capítulo 3 contrasta lo que el proyecto ya hace.

2.6 Estándares de calidad a nivel de producto

2.6.1 ISO/IEC 9126.

Publicado en 2001 y revisado en 2004, evalúa la calidad del producto a partir de seis características que se descomponen en subcaracterísticas y que se observan tanto de forma interna, sobre los atributos propios del producto, como externa, sobre su comportamiento al ejecutarse (ISO/IEC, 2001). Es el primer estándar que da una terminología común para hablar de calidad de producto, y por eso sus seis características siguen reconociéndose en los modelos posteriores.

Tabla 5

Las seis características de calidad de la norma ISO/IEC 9126

Característica	Subcaracterísticas
Funcionalidad	Adecuación, exactitud, interoperabilidad, seguridad de acceso y cumplimiento
Fiabilidad	Madurez, tolerancia a fallos, capacidad de recuperación y cumplimiento
Usabilidad	Capacidad de ser entendido, aprendido y operado, atractivo y cumplimiento
Eficiencia	Comportamiento temporal, utilización de recursos y cumplimiento
Mantenibilidad	Capacidad de ser analizado, cambiado y probado, estabilidad y cumplimiento

Característica	Subcaracterísticas
Portabilidad	Adaptabilidad, instalabilidad, coexistencia, reemplazabilidad y cumplimiento

Nota. Adaptado de Fillottrani, 2007 y ISO/IEC, 2001.

2.6.2 ISO/IEC 25000 SQuaRE.

La familia ISO/IEC 25000, conocida como SQuaRE por sus siglas en inglés, reemplaza y unifica las normas ISO/IEC 9126 e ISO/IEC 14598, y su objetivo es guiar el desarrollo de productos de software mediante la especificación y la evaluación de requisitos de calidad (ISO/IEC, 2005; ISO 25000, 2018). Su aporte es organizar en un solo marco lo que antes estaba repartido, pues define el modelo de calidad, la forma de medirlo, la manera de escribir los requisitos y el proceso para evaluarlos.

Tabla 6

Las cinco divisiones de la familia ISO/IEC 25000

División	Nombre	Qué contiene
ISO 2500n	Gestión de la calidad	Términos, definiciones y modelos de referencia comunes a toda la familia
ISO 2501n	Modelo de calidad	Las características y subcaracterísticas del producto y de la calidad en uso
ISO 2502n	Medida de la calidad	Las métricas con las que se cuantifica cada característica
ISO 2503n	Requisitos de calidad	Cómo se especifican los requisitos que después se van a evaluar
ISO 2504n	Evaluación de la calidad	El proceso de evaluación y los requisitos para quien evalúa

Nota. Adaptado de Alfonzo y Mariño, 2013 y ISO 25000, 2018.

2.6.3 ISO/IEC 25010.

Publicada en 2011 dentro de la división 2501n, reemplaza y actualiza el modelo de la norma ISO/IEC 9126 (ISO/IEC, 2011). Define dos modelos complementarios. El modelo de calidad del producto se compone de ocho características y treinta y una subcaracterísticas, y se refiere a las propiedades estáticas del software y dinámicas del sistema. El modelo de calidad en

uso se compone de cinco características y nueve subcaracterísticas, y describe el resultado de la interacción cuando el producto se emplea en un contexto particular (Polillo, 2011).

Tabla 7

Las ocho características del modelo de calidad de producto de la norma ISO/IEC 25010

Característica	Subcaracterísticas
Adecuación funcional	Compleitud, corrección y pertinencia funcional
Eficiencia de desempeño	Comportamiento temporal, utilización de recursos y capacidad
Compatibilidad	Coexistencia e interoperabilidad
Usabilidad	Capacidad de reconocer su adecuación, de aprendizaje y de ser usado, protección contra errores de usuario, estética de la interfaz y accesibilidad
Fiabilidad	Madurez, disponibilidad, tolerancia a fallos y capacidad de recuperación
Seguridad	Confidencialidad, integridad, no repudio, responsabilidad y autenticidad
Mantenibilidad	Modularidad, reusabilidad, analizabilidad, capacidad de ser modificado y de ser probado
Portabilidad	Adaptabilidad, facilidad de instalación y capacidad de ser reemplazado

Nota. Adaptado de ISO/IEC, 2011. Las cinco características del modelo de calidad en uso son efectividad, eficiencia, satisfacción, ausencia de riesgo y cobertura del contexto.

Frente a la norma ISO/IEC 9126, los cambios que más pesan para un sistema como este son dos. La seguridad deja de ser una subcaracterística escondida dentro de la funcionalidad y pasa a ser una característica propia con cinco subcaracterísticas, y aparece la compatibilidad, que es donde cae la capacidad de un sistema de operar junto a otros y de intercambiar información con ellos. Para una plataforma cuyo producto principal es una interfaz de integración consumida por sistemas ajenos, esas dos características no son un detalle de la actualización sino la razón para preferirla.

2.7 Estándares de calidad a nivel de proceso

2.7.1 ISO/IEC 15504 SPICE.

Es un marco de evaluación y mejora de los procesos de desarrollo y mantenimiento de software, más utilizado en Europa, que cubre tres áreas, la mejora de procesos, la evaluación de procesos y la determinación de la capacidad (ISO/IEC, 2004). Está alineado con la norma ISO/IEC 12207, que define los procesos del ciclo de vida del software (ISO/IEC, 2008), y provee

nueve documentos que guían la implementación del modelo y su evaluación. Su particularidad frente a CMMI es que se concentra en establecer un marco para evaluar antes que en prescribir un método de trabajo.

2.7.2 ISO 9001.

Pertenece a la familia ISO 9000 de sistemas de gestión de la calidad y se aplica a cualquier organización, no solo a las que desarrollan software (ISO, 2015). Su versión de 2015 se apoya en el enfoque a procesos, el pensamiento basado en riesgos y la orientación a las necesidades del cliente. Es un referente de gestión organizacional, de modo que aporta a la forma de trabajar de una empresa de desarrollo, pero no dice nada sobre los atributos que debe tener un programa, y esa distinción es la que se retoma en el capítulo siguiente.

3. El referente de calidad adoptado

De la revisión anterior se desprende que ningún referente cubre por sí solo lo que un proyecto necesita, porque los de producto no dicen nada sobre la forma de trabajar y los de proceso no dicen nada sobre los atributos del programa. FactuGest adopta en consecuencia dos referentes complementarios, la norma ISO/IEC 25010 para la calidad del producto y el modelo CMMI, en su nivel 2 de madurez, como referencia para el proceso.

3.1 Por qué la norma ISO/IEC 25010

La norma ISO/IEC 25010 es la versión vigente del modelo de calidad de producto y reemplaza a la norma ISO/IEC 9126 desde 2011 (ISO/IEC, 2011). Sobre esa condición de estándar actual pesan tres razones propias del sistema que se construyó.

La primera es que la seguridad dejó de ser una subcaracterística escondida dentro de la funcionalidad y pasó a ser una característica propia con cinco subcaracterísticas, entre ellas el no repudio y la responsabilidad. En un sistema que emite documentos con efectos legales por cuenta de otros, poder decir quién emitió qué y cuándo no es una precaución técnica sino parte del producto, y bajo la norma anterior ese requisito no tenía dónde clasificarse.

La segunda es la aparición de la compatibilidad como característica, con la interoperabilidad dentro. El producto principal de FactuGest es una interfaz que consume sistemas ajenos que nadie de este equipo escribió, de modo que la capacidad de intercambiar información con ellos no es un atributo secundario del sistema sino aquello por lo que el cliente paga.

La tercera es de método. La norma organiza la calidad en características y subcaracterísticas evaluables, lo que permite tomar los requisitos no funcionales que el proyecto ya había escrito y clasificarlos contra un referente externo, en lugar de inventar categorías

propias que solo se pueden juzgar por dentro. Ese cruce es el que se presenta a continuación, y de él sale además el orden en que se priorizaron las pruebas.

3.2 Los requisitos del sistema contra el modelo de calidad

El catálogo de requisitos no funcionales de FactuGest tiene 32 entradas, agrupadas por categorías que el equipo definió durante el análisis. Clasificarlas contra las ocho características de la norma permite comprobar dos cosas, si alguna característica del estándar quedó sin atender y si alguna categoría propia no encuentra lugar en el estándar. El resultado es que las ocho características están cubiertas, con un reparto que dice bastante sobre la naturaleza del sistema.

Tabla 8

Los requisitos no funcionales de FactuGest clasificados por característica de calidad

Característica	Requisitos	Qué se le exige al sistema	Con qué se sostiene hoy
Adecuación funcional	12, 13, 24	Que el documento diga lo que debe decir, que el consumo se cuente de lo realmente emitido y que lo emitido para un tercero no aparezca como venta propia	Las 35 pruebas de la aritmética tributaria y las 10 del membrete del PDF, más la separación en dos zonas de la base de datos
Eficiencia de desempeño	01, 02, 03, 30	Que la emisión responda en menos de tres segundos, que el envío del correo no demore la respuesta y que los listados no traigan la tabla completa	El correo agendado como tarea de fondo y las consultas paginadas del servicio de listados. La medición del tiempo bajo carga queda planificada
Compatibilidad	22, 23, 31	Que cualquier sistema pueda integrarse sin acuerdo previo y que un cambio no deje sin servicio a quien ya integró	El contrato publicado en formato OpenAPI, la colección de 11 peticiones de prueba y la política de sacar una versión nueva antes que romper la vigente
Usabilidad	18, 19, 20	Que la navegación se agrupe por trabajo y no por tabla, que los errores señalen el campo que los provocó y que la interfaz sirva en cualquier pantalla	Las 45 pruebas de las reglas de validación, que son las que producen el mensaje del formulario, y la encuesta aplicada a las empresas
Fiabilidad	11, 16, 17, 25	Que una emisión a medias no deje rastro, que la caída de un servicio externo no impida facturar y que lo emitido se conserve	La transacción que envuelve la emisión completa y el envío del correo fuera de esa transacción. El monitoreo de disponibilidad queda planificado

Característica	Requisitos	Qué se le exige al sistema	Con qué se sostiene hoy
Seguridad	04, 05, 06, 07, 08, 09, 10, 14, 15	Que ninguna credencial se guarde legible, que cada ruta exija sesión, que dos documentos no compartan número y que quede constancia de quién hizo qué	Las 11 pruebas del servicio de llaves, el registro de auditoría que solo se escribe, la reserva del consecutivo en una sola sentencia y el índice único por emisor, tipo y número
Mantenibilidad	26, 27, 28, 29	Que la aritmética exista una sola vez, que las rutas no contengan lógica, que el esquema cambie por migraciones y que haya pruebas	Las 190 pruebas automatizadas, la capa de servicios compartida por la aplicación web y la API, y el migrador que se puede ejecutar dos veces sin alterar nada la segunda
Portabilidad	21, 32	Que el sistema se use desde cualquier equipo y navegador sin instalar nada	La condición de aplicación web y la prueba en los navegadores de mayor uso

Nota. Las ocho características son las del modelo de calidad de producto de la norma ISO/IEC 25010 (ISO/IEC, 2011). Los códigos corresponden al catálogo de requisitos del entregable de requisitos funcionales y no funcionales del proyecto, donde consta el enunciado completo de cada uno y su medio de verificación.

La característica con más requisitos es seguridad, con 9 de 32, y el reparto no es casual.

Un proveedor tecnológico custodia las credenciales con las que otros emiten, numera documentos dentro de un rango autorizado y responde por lo que se hizo con esas credenciales, así que la mayor parte de lo que se le exige al sistema cae en confidencialidad, integridad, autenticidad, responsabilidad y no repudio. En un sistema de gestión corriente ese peso estaría en usabilidad o en adecuación funcional.

Tabla 9

Trazabilidad de cada requisito no funcional a la norma ISO/IEC 25010

Requisito	Categoría del catálogo	Característica	Subcaracterística
RNF 01	Rendimiento	Eficiencia de desempeño	Comportamiento temporal
RNF 02	Rendimiento	Eficiencia de desempeño	Comportamiento temporal
RNF 03	Rendimiento	Eficiencia de desempeño	Utilización de recursos
RNF 04	Seguridad	Seguridad	Confidencialidad
RNF 05	Seguridad	Seguridad	Confidencialidad
RNF 06	Seguridad	Seguridad	Confidencialidad
RNF 07	Seguridad	Seguridad	Autenticidad
RNF 08	Seguridad	Seguridad	Autenticidad
RNF 09	Seguridad	Seguridad	Confidencialidad e integridad
RNF 10	Integridad de los datos	Seguridad	Integridad
RNF 11	Integridad de los datos	Fiabilidad	Tolerancia a fallos

Requisito	Categoría del catálogo	Característica	Subcaracterística
RNF 12	Integridad de los datos	Adecuación funcional	Corrección funcional
RNF 13	Integridad de los datos	Adecuación funcional	Corrección funcional
RNF 14	Trazabilidad	Seguridad	Responsabilidad
RNF 15	Trazabilidad	Seguridad	No repudio
RNF 16	Disponibilidad	Fiabilidad	Tolerancia a fallos
RNF 17	Disponibilidad	Fiabilidad	Disponibilidad
RNF 18	Usabilidad	Usabilidad	Capacidad de ser usado
RNF 19	Usabilidad	Usabilidad	Protección contra errores de usuario
RNF 20	Usabilidad	Usabilidad	Accesibilidad
RNF 21	Compatibilidad	Portabilidad	Adaptabilidad
RNF 22	Compatibilidad	Compatibilidad	Interoperabilidad
RNF 23	Cumplimiento normativo	Compatibilidad	Interoperabilidad
RNF 24	Cumplimiento normativo	Adecuación funcional	Corrección funcional
RNF 25	Cumplimiento normativo	Fiabilidad	Capacidad de recuperación
RNF 26	Mantenibilidad	Mantenibilidad	Modularidad y reusabilidad
RNF 27	Mantenibilidad	Mantenibilidad	Modularidad
RNF 28	Mantenibilidad	Mantenibilidad	Capacidad de ser modificado
RNF 29	Mantenibilidad	Mantenibilidad	Capacidad de ser probado
RNF 30	Escalabilidad	Eficiencia de desempeño	Capacidad
RNF 31	Escalabilidad	Compatibilidad	Interoperabilidad
RNF 32	Portabilidad	Portabilidad	Adaptabilidad e instalabilidad

Nota. El enunciado completo de cada requisito y su medio de verificación están en el entregable de requisitos del proyecto. Aquí interesa la correspondencia entre la categoría con la que se escribió y la característica del estándar que le corresponde.

3.3 Donde el catálogo propio y el estándar no coinciden

El cruce dejó tres desajustes que conviene declarar, porque son justamente lo que aporta clasificar contra un referente externo en lugar de contra categorías propias.

- **Trazabilidad:** No existe como característica en la norma. Los dos requisitos que el catálogo agrupaba ahí, el registro de auditoría con el nombre del usuario y la imposibilidad de modificar ese registro, corresponden a responsabilidad y a no repudio, que son subcaracterísticas de seguridad. La norma los clasifica mejor de lo que estaban, porque deja explícito que su valor no es informativo sino probatorio

- **Escalabilidad:** Tampoco existe como característica, y sus dos requisitos se reparten.

Admitir nuevas empresas emisoras sin tocar el código es capacidad, dentro de eficiencia de desempeño, mientras que publicar una versión nueva de la interfaz antes que romper la vigente es interoperabilidad, dentro de compatibilidad

- **Compatibilidad de navegadores:** El catálogo lo puso bajo compatibilidad por el nombre, pero funcionar en cualquier navegador sin instalación es adaptabilidad, que pertenece a portabilidad. La compatibilidad de la norma se refiere a coexistir e intercambiar información con otros sistemas, que es lo que hace la interfaz de integración, no la interfaz de usuario

Ninguno de los tres obliga a reescribir el catálogo, que se escribió para el equipo y cumple su función. Lo que muestran es que un referente externo obliga a nombrar las cosas con un vocabulario que otros también usan, y esa es precisamente la utilidad de adoptar un estándar en lugar de declarar principios propios.

3.4 El referente de proceso, CMMI en su nivel 2

En el proceso, el referente adoptado es CMMI (CMMI Product Team, 2010), y la elección del nivel 2 es deliberada. El nivel 2 describe una organización que planifica y controla su trabajo por proyecto, gestiona sus requisitos y su configuración, mide lo que hace y asegura la calidad del producto y del proceso, que es exactamente el alcance que un equipo de formación puede sostener y demostrar. Declarar un nivel superior obligaría a mostrar procesos definidos para toda una organización, que aquí no existe.

Conviene además una precisión que evita una afirmación falsa. Un nivel de madurez se obtiene mediante una evaluación formal realizada por personal acreditado, de modo que este proyecto no está en el nivel 2 en el sentido de la certificación. Lo que se sostiene es que las siete

áreas de proceso de ese nivel se usaron como referencia y que cinco de ellas se cubren con evidencia verificable, mientras que dos se cubren de forma parcial.

Tabla 10

Las áreas de proceso del nivel 2 de CMMI frente al proyecto

Área de proceso	Estado	Con qué se sostiene
Gestión de requisitos	Cumplida	82 requisitos funcionales y 32 no funcionales catalogados, trazados a 56 casos de uso y, los críticos, a pruebas automatizadas
Planificación del proyecto	Cumplida	Trabajo por iteraciones bajo Scrum, con la pila de producto repartida en ocho sprints y un cronograma de quince actividades
Seguimiento y control del proyecto	Cumplida	Cuaderno de trabajo con bitácora de decisiones por tarea, y el tablero de control del propio sistema para la operación
Gestión de la configuración	Cumplida	Control de versiones sobre todo el código, migraciones de base de datos versionadas y registradas, y versionado explícito de la interfaz de integración
Aseguramiento de la calidad del proceso y del producto	Cumplida	Batería de pruebas ejecutable por cualquiera del equipo, invariantes del sistema escritos y revisión del formato de los entregables por herramienta
Medición y análisis	Parcial	Se mide el resultado de las pruebas y el consumo de los clientes, pero no hay todavía medición de tiempos de respuesta ni de densidad de defectos
Gestión de acuerdos con proveedores	Parcial	El acuerdo con el proveedor tecnológico habilitado está identificado y el adaptador está construido, pero la relación no se ha formalizado

Nota. Áreas de proceso según CMMI Product Team, 2010. El estado declarado como parcial corresponde a lo que existe pero no está completo, y no a lo que está planificado.

Las dos áreas parciales están declaradas como tales a propósito. En medición y análisis se mide el resultado de la batería de pruebas y el consumo de cada cliente, pero no hay todavía medición de tiempos de respuesta ni conteo de densidad de defectos, y eso es justamente lo que el plan de pruebas del capítulo siguiente deja como trabajo pendiente. En gestión de acuerdos con proveedores, la relación con el proveedor tecnológico habilitado ante la autoridad tributaria está identificada y el adaptador que la consumirá está construido y probado, pero el acuerdo no se ha formalizado, de modo que la transmisión real todavía no ocurre.

3.5 Los referentes que no se adoptaron

Elegir un referente obliga a decir por qué no se eligieron los otros, y en varios casos el motivo no es que sean peores sino que ya están recogidos en el que se adoptó.

Tabla 11

Referentes revisados y no adoptados como principales

Referente	Qué aporta	Por qué no se adoptó
Modelo de McCall	La partición de la calidad en operación, revisión y transición del producto	Varios de sus once factores no tienen métrica directa y terminan evaluándose con listas subjetivas. Su aporte ya está recogido, ordenado y ampliado en la norma vigente
Modelo de Boehm	La trazabilidad desde una característica general hasta la métrica que la mide	Es la idea que los estándares ISO adoptaron después. Adoptarlo hoy significaría usar una versión anterior de lo mismo
ISO/IEC 9126	Las seis características que fundaron el vocabulario común de calidad de producto	Fue reemplazada en 2011 por la norma ISO/IEC 25010. Declarar como referente una norma sustituida es un defecto que un evaluador señala de inmediato
ISO/IEC 15504 SPICE	Un marco de evaluación de procesos por niveles de capacidad, alineado con el ciclo de vida del software	Exige una evaluación formal por perfiles de capacidad que este proyecto no va a ejecutar. Declararlo sin evaluarlo sería afirmar algo que no se puede demostrar
ISO 9001	El enfoque a procesos, el pensamiento basado en riesgos y la orientación al cliente	Norma la gestión de una organización y no los atributos de un programa. FactuGest es un proyecto formativo sin sistema de gestión de la calidad certificable

Nota. La revisión de cada uno está en el capítulo 2. Las normas ISO/IEC 15504 e ISO 9001 se citan según ISO/IEC, 2004 e ISO, 2015.

Queda una precisión sobre la norma ISO/IEC 15504. Que no se adopte como marco de evaluación no significa que su contenido sea ajeno al proyecto, pues está alineada con la norma ISO/IEC 12207 sobre los procesos del ciclo de vida (ISO/IEC, 2008), y varios de esos procesos, como la verificación y la gestión de la configuración, son los mismos que el nivel 2 de CMMI pide y que el proyecto sí aplica. La diferencia está en el instrumento, no en el fondo, y Scalone

(2006) ya señalaba que los referentes de proceso comparten buena parte de sus prácticas y se distinguen sobre todo por la forma de evaluarlas.

4. Plan de pruebas

El plan que sigue documenta cómo se verificó FactuGest. Se organiza con la estructura habitual de la documentación de pruebas, con su objetivo, su alcance, su entorno, sus criterios y sus riesgos (IEEE, 2008; ISO/IEC/IEEE, 2013), y describe lo que el proyecto hizo, de modo que cada nivel declarado como ejecutado tiene su evidencia en el capítulo siguiente y cada nivel que no se ejecutó aparece como planificado.

4.1 Objetivo

El objetivo es comprobar que FactuGest emite documentos electrónicos correctos en contenido, en cálculo y en identificación del emisor, que la interfaz de integración cumple el contrato que publica, y que las reglas de acceso y de registro de la actividad se aplican. Dicho de otro modo, no se busca demostrar que el sistema funciona sino encontrar aquello en lo que falla, que es la diferencia entre probar y confirmar lo que uno ya cree (Myers et al., 2011).

4.2 Alcance

Entra en el alcance todo lo que produce o modifica un documento electrónico y todo lo que decide quién puede hacerlo. En concreto, la aritmética tributaria con sus bases, descuentos y prorrateo del impuesto, la reserva del consecutivo autorizado, la generación de la representación gráfica y del archivo de intercambio, el contrato de entrada y de salida de la interfaz de integración, el control de acceso por rol, el registro de la actividad y el cálculo del consumo de cada cliente contra el cupo de su plan.

Queda fuera del alcance, y conviene decirlo antes de que alguien lo busque en el capítulo de resultados, la transmisión real ante la autoridad tributaria, que depende de un proveedor tecnológico habilitado (DIAN, 2020) con el que todavía no se ha formalizado el acuerdo. Queda fuera también el comportamiento del sistema bajo carga concurrente alta, porque no se ha

ejecutado la prueba que lo mediría, y la verificación del despliegue en un servidor de producción, que todavía no existe.

La prioridad dentro del alcance no se repartió por igual. Sale del cruce del capítulo anterior, donde la característica con más requisitos resultó ser la seguridad, seguida por la eficiencia de desempeño, la fiabilidad y la mantenibilidad (ISO/IEC, 2011). A eso se suma un criterio de daño, pues lo que se prueba primero es aquello cuyo fallo produce un documento que no se puede corregir, como un número de resolución repetido, un impuesto mal calculado o una factura que dice haber sido expedida por quien no la expidió.

4.3 Estrategia y niveles de prueba

Se aplicaron cuatro niveles de prueba y cuatro tipos adicionales, con la distinción habitual entre lo que se verifica en cada uno (ISTQB, 2018). La estrategia se apoya en dos decisiones que conviene explicar, porque determinan qué se automatizó y qué no.

La primera es que se automatiza lo que no necesita base de datos. La aritmética, las reglas de validación, el contrato de la interfaz, la generación del archivo de intercambio y el armado de la representación gráfica son funciones que reciben datos y devuelven un resultado, de modo que pueden probarse miles de veces en segundos y sirven de red permanente. Lo que exige base de datos y sesión de usuario se prueba de forma funcional, sobre el sistema en marcha, con registro en imagen del antes y el después.

La segunda es que el resultado esperado de una prueba se calcula a mano y no se copia de la salida del programa. Una prueba que repite lo que el código devolvió no verifica nada, porque queda en verde tanto si el código está bien como si está mal, y esa es una trampa fácil de caer cuando se escriben pruebas después del código y no antes (Beck, 2003).

Tabla 12*Niveles y tipos de prueba aplicados*

Nivel o tipo	Qué verifica	Técnica	Con qué se ejecuta	Estado
Unitaria	Una función o un servicio aislado, sin base de datos ni servidor	Caja blanca, con partición de equivalencia y valores límite	pytest	Ejecutada
Integración	El recorrido completo de la interfaz de integración, desde la llave hasta la nota crédito	Caja negra sobre el contrato publicado	Colección de peticiones y documentación interactiva	Ejecutada
Sistema y funcional	Las operaciones del panel de principio a fin, con la base de datos real	Caja negra, por caso de uso	Navegador, con registro en imagen	Ejecutada
Aceptación	Que lo entregado corresponda a lo que el requisito pedía	Revisión contra el catálogo de requisitos y los casos de uso	Revisión del equipo	Ejecutada
Usabilidad	Que la navegación y los mensajes sirvan a quien no construyó el sistema	Encuesta a empresas y observación directa	Instrumento de encuesta	Ejecutada
Seguridad	El almacenamiento de credenciales, el control de acceso por rol y el cierre de sesión	Inspección y prueba dirigida	Revisión de la base y pruebas automatizadas	Ejecutada en parte
Rendimiento	El tiempo de respuesta de la emisión bajo carga concurrente	Prueba de carga	Herramienta de carga, por definir	Planificada
Regresión	Que una corrección no rompa lo que ya funcionaba	Nueva ejecución de la batería completa	pytest, hoy manual	Ejecutada de forma manual

Nota. Clasificación de niveles según ISTQB, 2018. El estado corresponde a lo ocurrido hasta la fecha de este informe.

El volumen resultante es de 190 pruebas automatizadas repartidas en 12 archivos, 11 peticiones de prueba sobre la interfaz de integración y 4 operaciones verificadas de forma funcional con registro en imagen. El detalle de cada grupo, con lo que protege y su resultado, está en el capítulo 5.

4.4 Entorno y datos de prueba

Las pruebas se ejecutan en el entorno de desarrollo de cada integrante, que reproduce el del sistema completo. No hay todavía un servidor de pruebas separado, lo que se declara como riesgo más adelante.

Tabla 13

Entorno de ejecución de las pruebas

Componente	Qué se usa
Sistema operativo	Windows 11
Lenguaje	Python 3.13.3
Marco de trabajo web	FastAPI, servido con Uvicorn
Base de datos	MySQL, con el esquema aplicado por el migrador del proyecto
Ejecutor de pruebas	pytest 9.1.1
Pruebas de la interfaz de integración	Colección de peticiones y la documentación interactiva que publica el propio servidor
Pruebas del panel	Navegador de escritorio
Datos	Sembrador determinista del proyecto

Nota. La batería automatizada se ejecuta con pytest 9.1.1 sobre Python 3.13.3 y su corrida completa tarda 3,39 segundos.

Los datos merecen una explicación aparte, porque de ellos depende que una prueba signifique algo. El proyecto tiene un sembrador que crea la empresa emisora, el catálogo de servicios, catorce empresas suscritas con su llave y su plan, tres meses de documentos emitidos por cuenta de ellas y seis meses de ventas propias. Tiene tres propiedades pensadas para pruebas. Es determinista, de manera que dos siembras producen exactamente lo mismo y un resultado distinto señala un cambio en el sistema y no en los datos. Se verifica a sí mismo al terminar. Y sabe deshacer con exactitud lo que creó, de modo que la base queda como estaba.

Las pruebas automatizadas, en cambio, no usan el sembrador ni tocan la base de datos, y esa fue una decisión del plan. Una prueba que depende de una base depende también de su estado, del orden en que se ejecuta y de que la base esté levantada, con lo cual empieza a fallar

por motivos que no tienen que ver con lo que pretende verificar. Las funciones que sí necesitan la base se verifican en el nivel funcional, con el sistema en marcha.

4.5 Criterios de entrada y de salida

Una tarea entra a prueba cuando cumple estas condiciones.

- El requisito que la origina está escrito en el catálogo del proyecto y tiene un medio de verificación declarado.
- El código está integrado en la rama de trabajo y el sistema levanta sin errores.
- Los datos necesarios existen, sea porque los crea el sembrador o porque la prueba los construye.

Y se considera probada cuando se cumplen todas las siguientes, que son las que cierran una tarea y permiten pasar a la que sigue.

- La batería completa de 190 pruebas automatizadas termina sin ninguna falla, y no solamente las pruebas del módulo que se tocó.
- Las peticiones de la interfaz de integración responden con el código y la forma que documenta el contrato, incluido el caso del error.
- La operación se ejecutó en el panel y quedó su registro en imagen, cuando se trata de una función de la aplicación web.
- No queda ningún defecto de severidad crítica abierto, y los de severidad mayor están registrados con su decisión.

4.6 Gestión de los defectos

Un defecto encontrado sigue siempre el mismo recorrido, y el orden de los pasos importa. Primero se escribe una prueba que lo reproduce y que, por lo tanto, falla. Después se corrige el código hasta que esa prueba pasa. Y por último la prueba se queda en la batería, de modo que el

defecto no puede volver sin que alguien se entere. Corregir primero y probar después deja la duda de si la prueba habría detectado el defecto original, que es precisamente lo que se necesita saber.

Los defectos se clasifican por lo que producen y no por lo que cuesta arreglarlos, porque lo que decide si una entrega se detiene es el daño y no el esfuerzo.

Tabla 14

Clasificación de los defectos por severidad

Severidad	Qué la define	Ejemplo encontrado en el proyecto
Crítica	Produce un documento inválido ante la autoridad tributaria, expone datos o impide facturar	El PDF de un documento emitido por la interfaz de integración salía con el nombre y el logo del proveedor y no con los de la empresa que expedía
Mayor	El documento sale, pero dice algo que no se puede explicar o falta un dato obligatorio	El pie de la factura mostraba un descuento del 0,0 % restando dinero real, y la ciudad del comprador salía vacía en todo lo emitido por la interfaz
Menor	El contenido es correcto pero su presentación estorba la lectura	Las columnas de dinero partían los importes de ocho cifras, de modo que el impuesto quedaba con un dígito solitario en el renglón siguiente

Nota. Los tres ejemplos son defectos reales encontrados durante la construcción y ya corregidos. Su detalle y la prueba que hoy los cubre están en el capítulo 5.

4.7 Riesgos del plan

Todo plan de pruebas tiene puntos ciegos, y declararlos es parte del plan. Los cinco siguientes son los de este, con lo que se hizo para reducirlos.

Tabla 15

Riesgos del plan de pruebas y su tratamiento

Riesgo	Qué pasa si ocurre	Cómo se atiende
El entorno de pruebas no es el de producción	Un defecto que solo aparece con concurrencia real o con volumen alto no se detecta antes de desplegar	Los invariantes que dependen de la concurrencia, como la reserva del consecutivo, se resolvieron por diseño y no por prueba, con una sola sentencia y un índice único que la base hace cumplir
No hay transmisión real ante la autoridad tributaria	El comportamiento del proveedor habilitado podría diferir de lo que supone el adaptador	El adaptador tiene 24 pruebas que fijan el contrato esperado, de modo que la diferencia se detecta el primer día de la integración real y no en producción

Riesgo	Qué pasa si ocurre	Cómo se atiende
El equipo es pequeño y no hay un probador independiente	Quien escribió el código tiende a probar lo que pensó y no lo que puede fallar	Las pruebas se escriben con valores calculados a mano y no copiados de la salida del programa, y cada integrante revisa lo que escribió otro
Los datos de prueba son sembrados	Un caso que la realidad produce y el sembrador no, queda sin probar	El sembrador reproduce la operación de catorce empresas durante varios meses, y las pruebas unitarias no dependen de él porque no tocan la base de datos
No hay ejecución automática en cada cambio	Una corrección puede romper algo ya probado y nadie enterarse hasta la siguiente corrida	La batería completa tarda segundos y se ejecuta antes de cerrar cada tarea. La ejecución automática queda planificada

Nota. Ninguno de los cinco está resuelto por completo. Los que dependen del despliegue se atienden en la fase correspondiente del proyecto.

El primero merece una nota, porque es el que más se subestima. Hay defectos que una prueba no puede encontrar en un entorno de un solo usuario, y el más caro de este sistema es el de dos peticiones simultáneas que leen el mismo consecutivo y emiten dos documentos con el mismo número de una resolución autorizada. Ese riesgo no se cubrió con una prueba sino por diseño, reservando el número en una sola sentencia y dejando además un índice único en la base que la propia base hace cumplir. Es la respuesta correcta, porque una prueba de concurrencia detecta el problema algunas veces, mientras que el índice lo impide siempre.

4.8 Roles y responsabilidades

El equipo es de tres personas, de modo que los roles no corresponden a integrantes distintos sino a funciones que cada uno asume según la tarea. La regla que sí se sostiene es que nadie ejecuta la prueba funcional de lo que él mismo escribió.

Tabla 16

Roles del plan de pruebas

Rol	Responsabilidad
Analista de pruebas	Deriva los casos del catálogo de requisitos y decide qué se prueba primero según el riesgo
Desarrollador	Escribe la prueba unitaria junto con el código que la satisface y la deja en la batería
Probador	Ejecuta las pruebas funcionales sobre lo que escribió otro integrante y levanta el registro en imagen

Rol	Responsabilidad
Responsable de la corrección	Reproduce el defecto con una prueba que falla, lo corrige y deja esa prueba como regresión

Nota. Los cuatro roles rotan entre los tres integrantes del equipo.

Como referencia del trabajo cubierto, el catálogo del proyecto tiene 32 requisitos no funcionales con su medio de verificación declarado, y todos ellos quedaron clasificados en el capítulo anterior. La colección de peticiones de prueba está publicada en el propio repositorio (Factugest/docs/FactuGest.postman_collection.json) y las imágenes de las pruebas funcionales, en docs expo/documento para que te guíes claude.

5. Pruebas ejecutadas

Este capítulo presenta lo que se ejecutó y su resultado. La batería automatizada se corrió por última vez el 8 de septiembre de 2026 con pytest 9.1.1 sobre Python 3.13.3, y sus 190 pruebas terminaron sin ninguna falla en 3,39 segundos. La salida completa de esa corrida va como anexo al final del informe, de modo que las cifras que siguen no dependen de que el lector confíe en el texto.

5.1 Pruebas unitarias

Las pruebas unitarias cubren lo que se puede verificar sin base de datos ni servidor, que en este sistema es justamente donde vive el riesgo mayor, pues ahí están la aritmética con la que se calcula un impuesto, las reglas con las que se acepta o se rechaza un dato y el armado de los dos archivos que constituyen el documento electrónico.

Tabla 17

Reparto de las pruebas unitarias por lo que protegen

Archivo de pruebas	Pruebas	Qué protege
test_validaciones.py	45	Las reglas de validación del dominio, desde el formato de un correo hasta el dígito de verificación del NIT y el orden de las fechas de una resolución
test_dian_proveedor.py	24	El adaptador del proveedor de facturación electrónica, que además deja escrito el contrato que tendrá que cumplir el proveedor real
test_modelos_api.py	22	El contrato de entrada de la API, comprobando que aplica las mismas reglas que el formulario web y no unas propias
test_calculo_documento.py	19	La aritmética tributaria, con bases gravables, descuentos de línea y de factura y prorratio del impuesto
test_calculo_notas.py	16	La aritmética de las notas crédito y débito, que es donde un redondeo mal puesto se convierte en dinero a favor o en contra de alguien
test_monograma.py	13	El distintivo de la empresa que no cargó logo, que debe ser siempre el mismo para el mismo nombre
test_api_key_service.py	11	La generación y el formato de las llaves de la API, con sus propiedades criptográficas
test_pdf_membrete.py	10	Que el membrete del PDF sea el del emisor del documento y nunca el de FactuGest
test_consumo_service.py	9	El cálculo del excedente sobre el cupo del plan y el aviso previo al cliente

Archivo de pruebas	Pruebas	Qué protege
test_xml_service.py	9	El XML bajo el estándar UBL 2.1 en los puntos donde el emisor cambia la salida
test_documento_canonico.py	6	El contrato de entrada que comparten el generador de PDF y el de XML
test_pdf_descuentos.py	6	Que cada descuento salga en el pie de la factura con el porcentaje de la base que de verdad le corresponde

Nota. Total de 190 pruebas en 12 archivos. La cantidad sale de la corrida y no de contar funciones, porque una prueba escrita una vez y ejecutada con varios juegos de datos cuenta por cada juego.

El reparto no es proporcional al tamaño del código sino al daño que produce un error. Las reglas de validación y la aritmética tributaria concentran la mayor parte porque un dato mal aceptado o un impuesto mal calculado terminan impresos en un documento con efectos legales (DIAN, 2020), mientras que un error en una pantalla de consulta se corrige recargando la página.

5.2 Casos de prueba y sus resultados

La tabla siguiente detalla quince casos representativos de los distintos grupos. No son ejemplos redactados para este informe, sino pruebas de la batería con sus valores tal como están escritos, y por eso los importes son los que son y no cifras redondas. Los quince resultaron superados, que es lo esperable de una batería que se ejecuta a diario, y su valor no está en el estado sino en lo que cada uno impide que vuelva a pasar.

Tabla 18*Casos de prueba representativos y sus resultados*

Caso	Qué verifica	Entrada	Resultado esperado	Resultado obtenido
CP-01	El cálculo de una línea con impuesto	2 unidades a 189.000 con tarifa del 19 %	Bruto de 378.000 e impuesto de 71.820	Bruto de 378.000 e impuesto de 71.820
CP-02	El descuento propio de una línea	2 unidades a 189.000, con 5 % de descuento y tarifa del 19 %	Descuento de 18.900, base de 359.100 e impuesto de 68.229	Descuento de 18.900, base de 359.100 e impuesto de 68.229
CP-03	Que una línea exenta no genere impuesto	3 unidades a 12.000 con tarifa del 0 %	Base de 36.000 e impuesto de 0	Base de 36.000 e impuesto de 0
CP-04	Un documento con tarifas distintas en sus líneas	Tres líneas, al 19 %, al 5 % con 10 % de descuento y al 0 %	Bruto de 162.000, descuentos de 3.000, base de 159.000, impuestos de 20.350 y total de 179.350	Los cinco valores coinciden
CP-05	El prorrateo del impuesto sobre el descuento de factura	Una línea de 100.000 al 19 % con descuento de factura de 10.000	Base de 90.000 e impuesto de 17.100, y no los 19.000 de la base sin descuento	Base de 90.000 e impuesto de 17.100
CP-06	El prorrateo cuando el documento tiene varias tarifas	Las tres líneas de CP-04 con descuento de factura de 9.000	Descuentos de 12.000, base de 150.000, impuestos de 19.198,11 y total de 169.198,11	Los cuatro valores coinciden
CP-07	Que una nota crédito de anulación devuelva lo facturado	Factura de tres teclados con 10 % de descuento y un ratón, todo al 19 %	La nota repite la base, los impuestos y el total de la factura, y su mismo número de líneas	Los tres importes y el número de líneas coinciden
CP-08	Que una devolución parcial prorratee base, descuento e impuesto	Devolución de una unidad de las tres de la primera línea	Bruto de 100.000, descuento de 10.000, base de 90.000, impuesto de 17.100 y total de 107.100, que es un tercio exacto	Los cinco valores coinciden
CP-09	El dígito de verificación del NIT según el algoritmo de la autoridad	Los NIT 890903938, 800197268 y 900373115	Los dígitos 8, 4 y 3 respectivamente	Los tres dígitos coinciden
CP-10	Que la identificación se valide según su tipo de documento	El valor AB1234 como cédula y el valor AV123456 como pasaporte	La cédula se rechaza indicando que solo admite números, y el pasaporte se acepta	El rechazo y la aceptación se producen
CP-11	El excedente sobre el cupo del plan	418 documentos emitidos contra un cupo de 400, y 400 contra 400	Excedente de 18 en el primer caso y de 0 en el segundo, con el aviso encendido	Excedente de 18 y de 0, con el aviso encendido

Caso	Qué verifica	Entrada	Resultado esperado	Resultado obtenido
CP-12	Las propiedades de la llave de la interfaz de integración	Cincuenta llaves generadas de forma consecutiva	Prefijo de veinte caracteres como máximo, secreto de treinta como mínimo, ninguna llave repetida y un resumen que no permite recuperar el secreto	Las cuatro propiedades se cumplen
CP-13	Que el membrete del documento sea el de la empresa que emite	Documento de una clínica generado con su emisor	El texto del documento no contiene en ninguna parte el nombre del proveedor tecnológico	El nombre del proveedor no aparece
CP-14	El rechazo de un documento cuyo total no cuadra	Cabecera con un total de 999.999 que no corresponde a su base más sus impuestos	Rechazo explicando la inconsistencia, devuelto como respuesta y no como error del programa	Rechazo con el mensaje esperado y sin excepción
CP-15	Que emitir dos veces la misma venta no gaste dos números	La misma petición de emisión enviada por segunda vez con su misma referencia	La segunda responde con el documento que ya existía y no crea uno nuevo	Devuelve el documento existente

Nota. Los quince casos quedaron superados. Los valores esperados están calculados a mano en la propia prueba y no copiados de la salida del programa, que es la única forma de que una prueba distinga un cálculo correcto de uno que solo es consistente consigo mismo.

Tres casos merecen un comentario. El CP-05 y el CP-06 verifican el prorrateo del impuesto cuando la factura lleva un descuento global, que es el punto donde más fácil se equivoca un facturador, porque calcular el impuesto sobre la base sin descontar produce un documento que cobra de más y calcularlo aparte por línea produce diferencias de redondeo que no cuadran con el total. El CP-08 comprueba que una devolución parcial reparta también el descuento, ya que devolver la base completa de una línea que se vendió con rebaja significa regresar más dinero del que se recibió.

5.3 Pruebas de integración sobre la interfaz

La interfaz de integración se probó con una colección de 11 peticiones que recorre el ciclo completo de una integración, en el mismo orden en que lo haría el sistema de un cliente. La colección está publicada en el repositorio del proyecto y sirve a la vez de prueba y de documentación para quien vaya a integrarse, de modo que no es un material escrito para el informe.

Tabla 19

Recorrido de las pruebas sobre la interfaz de integración

N.º	Petición	Método y ruta	Respuesta esperada	Qué se comprueba
1	Comprobar la llave	GET /api/v1/ping	200	El emisor con el que numera quien integra
2	Emitir una factura	POST /api/v1/facturas	201	El número, el código único y los enlaces a los dos archivos
3	Reintentar la misma venta	POST /api/v1/facturas	200	El mismo documento de la primera emisión, sin número nuevo
4	Consultar un documento	GET /api/v1/documentos/{id}	200	El estado actual del documento y si una nota lo anuló
5	Descargar el PDF	GET /api/v1/documentos/{id}/pdf	200	Un archivo PDF
6	Descargar el XML	GET /api/v1/documentos/{id}/xml	200	El archivo de intercambio en el estándar UBL 2.1

N.º	Petición	Método y ruta	Respuesta esperada	Qué se comprueba
7	Listar los documentos del mes	GET /api/v1/documentos	200	El listado paginado con la referencia con la que se envió cada uno
8	Nota crédito, anular la factura	POST /api/v1/notas-credito	201	La nota con los importes de la factura original
9	Nota crédito, devolución parcial	POST /api/v1/notas-credito	201	La parte proporcional de base, descuento e impuesto
10	Nota débito, cobrar un adicional	POST /api/v1/notas-debito	201	El adicional con el impuesto descompuesto o sumado, según se pida
11	Un error, para ver su forma	GET /api/v1/ping	401	La forma común de los errores, con su código y su mensaje

Nota. Las once respondieron con el código y la forma esperados. La petición 3 es la que comprueba que repetir una emisión no gasta un segundo número de la resolución, y la 11 provoca un error a propósito para verificar que todos salen con la misma forma.

La petición número 3 es la más importante de la colección y conviene explicar por qué.

Un sistema que integra puede perder la respuesta de una emisión por un corte de red y volver a enviarla, y si esa segunda petición emitiera otro documento, el cliente habría gastado dos números de una resolución autorizada para una sola venta. La prueba envía dos veces la misma petición y verifica que la segunda devuelva el documento que ya existía.

5.4 Pruebas funcionales

Las operaciones que necesitan base de datos y sesión se verificaron sobre el sistema en marcha, ejecutando la operación completa desde el navegador y dejando registro del formulario diligenciado y del resultado. Se cubrieron 4 operaciones, cada una con su par de imágenes.

Tabla 20

Operaciones verificadas de forma funcional

Operación	Qué se comprueba	Resultado
Registrar un servicio del catálogo	Que el servicio quede disponible para facturarse	El servicio aparece en el catálogo y puede seleccionarse en una factura
Registrar un cliente	Que se validen la identificación y la ubicación contra los catálogos	El cliente queda registrado con su municipio y su departamento
Emitir una factura	Que la factura tome el consecutivo, calcule los totales y genere sus archivos	La factura queda emitida con su número, su código único y su representación gráfica

Operación	Qué se comprueba	Resultado
Registrar un usuario	Que la cuenta quede creada con su rol y su contraseña cifrada	El usuario puede ingresar y ve solo lo que su rol permite

Nota. Cada operación se registró en dos imágenes, el formulario diligenciado y el resultado. Las ocho imágenes están en el repositorio del proyecto.

Figura 1

Formulario de emisión de una factura, diligenciado antes de emitir

FACTUGEST

Nueva factura de venta
Emite factuGest S.A.S. - NIT 901245678-1

Ver formulario anterior Cancelar Emitir factura

1 ¿A quién le facturas?
Busca por nombre o por documento

Papelería Escolar Cúcuta
NIT 1090345678 Cambiar

2 ¿Qué le vendes?
Escribe y presiona Enter para agregar

Nombre o código del producto...

CONCEPTO	CANTIDAD	PRECIO	DESCUENTO	TOTAL
Plan Pro — 400 documentos/mes PLAN-PRO	1	\$189,000	—	\$224,910

3 ¿Cómo paga?
De contado o a crédito, y con qué medio

FORMA DE PAGO: De contado (Paga ahora), A crédito (Paga después)

MEDIO DE PAGO: CASH, DEBIT CARD, CREDIT CARD, BANK TRANSFER

ESTADO DE LA FACTURA: paid, cambiar

OBSERVACIONES (salen impresas):

DESCUENTO A TODA LA FACTURA:

Resumen 1 concepto

Subtotal	\$189,000
Descuentos	-\$0
Base gravable	\$189,000
IVA	\$35,910
Total	\$224,910

Todo listo para emitir

Emitir factura

Una vez emitida, la factura solo se corrige con una nota crédito.

Nota. La operación se ejecuta con el sistema en marcha y sobre la base de datos real, que es lo que distingue una prueba funcional de una unitaria.

Figura 2

Resultado de la emisión, con la factura ya registrada



FactuGest S.A.S.
NIT. 901245678-1 RESPONSABLE IVA
No somos Automatenedores
Av. Gran Colombia # 12-45, oficina 402 - San José De Cucuta
Teléfonos: 6075832100 Web: factugest.co Contacto: facturacion@factugest.co
RESOLUCION DIAN No. 18764002451009 DEL 01/01/2026 AUTORIZA DEL FG 1 AL FG
5000 FECHA VIGENCIA DESDE 01/01/2026 HASTA 31/12/2026

Factura Electrónica de Venta
FG74
Fecha de Generación: 20/08/2026, 7:42 PM
Fecha de Expedición: 20/08/2026, 7:42 PM
Fecha de Vencimiento: 20/08/2026

Cliente: PAPELERÍA ESCOLAR CÚCUTA Nº Identificación: NIT 1090345678 Dirección: CALLE 6 # 3-21, BARRIO LATINO Ciudad: SAN JOSÉ DE CÚCUTA Departamento: NORTE DE SANTANDER País: COLOMBIA Teléfono: 3156644821 Correo: PAPELERIAESCOLAR54@GMAIL.COM	Nombre Vendedor: Orden de Compra: Forma de Pago: CONTADO Medio de Pago: CREDIT CARD
--	--

ID	CÓDIGO PRINCIPAL	DESCRIPCIÓN	CANTIDAD	UNID MEDIDA	PRECIO UNITARIO	BRUTO	%IVA	IVA	TOTAL
1	PLAN-PRO	Plan Pro — 400 documentos/mes	1.00	MON	189,000.00	189,000.00	19.00	35,910.00	224,910.00



Total de Líneas	1
Bruto / Subtotal	\$ 189,000.00
Base Gravable	\$ 189,000.00
IVA	\$ 35,910.00
Moneda	COP
Total	\$ 224,910.00

SON: DOSCIENTOS VEINTICUATRO MIL NOVECIENTOS DIEZ PESO(S) CON CERO CENTAVO(S)

Observaciones:

CUFE: c60503281a7a24b5729f6450d4145128f95d9e657126d3f85c2a35059cee10691e1eb93f29590f44606010e1a2112 Página 1 de 1

Nota. El documento queda con su número tomado del consecutivo autorizado, sus totales calculados y sus archivos generados.

5.5 Pruebas de seguridad

La seguridad es la característica con más requisitos del sistema, según el cruce del capítulo 3 (ISO/IEC, 2011), de modo que se verificó punto por punto. Lo que sigue es lo comprobado, que corresponde a inspección y prueba dirigida y no a un análisis con herramienta especializada, el cual queda planificado en el capítulo siguiente.

Tabla 21

Verificaciones de seguridad ejecutadas

Qué se verificó	Cómo	Resultado
-----------------	------	-----------

Qué se verificó	Cómo	Resultado
Que ninguna contraseña quede legible	Inspección de la tabla de usuarios después de crear una cuenta	Solo consta el resumen calculado con el algoritmo de cifrado en un sentido
Que del secreto de una llave solo se guarde su resumen	Inspección de la tabla de clientes y las pruebas del servicio de llaves	En claro solo queda el prefijo, que sirve para localizar la fila
Que toda ruta del panel exija sesión	Petición a una ruta protegida sin haber iniciado sesión	Redirección al formulario de ingreso
Que el acceso se limite por rol	Ingreso con una cuenta de cajero a las rutas reservadas	Bloqueo de los módulos de la plataforma, los reportes y la configuración
Que la sesión se cierre por inactividad y no por tiempo transcurrido	Uso continuo frente a inactividad prolongada durante el mismo lapso	El uso continuo no corta la sesión y la inactividad sí
Que una llave inválida y una suspendida se distingan	Peticiones con una llave mal formada y con una de un cliente suspendido	Respuesta 401 en el primer caso y 403 en el segundo
Que no entren credenciales al registro de actividad	Rotación de una llave y revisión del registro	Consta el hecho de la rotación y no la llave

Nota. La distinción entre la respuesta 401 y la 403 no es un detalle técnico, pues en la primera quien integra revisa su configuración y en la segunda tiene que comunicarse con el proveedor.

5.6 Pruebas de usabilidad

La usabilidad se verificó con dos técnicas distintas y en dos momentos. Durante el análisis se aplicó una encuesta a empresas de la ciudad para conocer cómo facturaban y qué dificultades tenían, y esa información orientó decisiones de interfaz, en particular la de agrupar el menú por trabajo y no por tabla y la de dejar un solo camino hacia cada destino, pues la versión anterior repetía cuatro enlaces en dos sitios.

Durante la construcción se aplicó observación directa sobre el formulario de emisión, que es la pantalla que más se usa y la única que un cajero utiliza a diario. De ahí salió su división en tres pasos, a quién se factura, qué se factura y cómo se paga, con el resumen de los totales siempre visible. La medición formal de la satisfacción, con un instrumento aplicado después del uso, queda planificada y se declara como tal (ISTQB, 2018).

6. Pruebas planificadas

Lo que sigue no se ha ejecutado y se declara con esas palabras. La guía admite presentar pruebas ejecutadas o planificadas, y presentar como ejecutado lo que no se hizo invalidaría también lo que sí, porque un informe de pruebas vale exactamente lo que valga su parte menos verificable.

Cada prueba planificada se escribe con su métrica de aprobación y con lo que hace falta para poder ejecutarla (ISO/IEC/IEEE, 2013). Sin métrica no hay forma de saber si una prueba de rendimiento salió bien, y sin la condición de inicio la planificación se convierte en una lista de buenas intenciones que nadie puede programar.

Tabla 22

Pruebas planificadas, su métrica de aprobación y su condición de inicio

Prueba	Qué se mediría	Métrica de aprobación	Qué hace falta
Carga sobre la emisión	El tiempo de respuesta de la emisión con varias peticiones simultáneas	El 95 % de las emisiones por debajo de tres segundos, sin ninguna respuesta errónea ni ningún consecutivo repetido	Un servidor de pruebas separado del equipo de desarrollo y una herramienta de carga
Concurrencia sobre la numeración	Que dos peticiones simultáneas del mismo emisor no obtengan el mismo número	Ningún número repetido en varias emisiones lanzadas a la vez	El mismo servidor de pruebas. Hoy el riesgo está cubierto por diseño, con la reserva en una sola sentencia y el índice único
Análisis de dependencias	Las bibliotecas de terceros que usa el sistema y sus vulnerabilidades conocidas	Ninguna vulnerabilidad de severidad alta sin tratar	Una herramienta de análisis y la decisión de con qué periodicidad se corre
Pruebas dirigidas de entrada	El comportamiento de la interfaz ante datos malformados, desbordados o maliciosos	Ninguna entrada que provoque un error del programa en lugar de un rechazo controlado	Ampliar la colección de peticiones con los casos de abuso
Regresión automática	Que la batería completa se ejecute sola con cada cambio integrado	Ningún cambio integrado con la batería en rojo	Un servicio de integración continua sobre el repositorio

Prueba	Qué se mediría	Métrica de aprobación	Qué hace falta
Validación con el proveedor habilitado	Que un documento real sea aceptado por la autoridad tributaria	Acuse de recibo y aceptación del documento transmitido	El acuerdo formalizado con un proveedor tecnológico habilitado
Usabilidad después del uso	La satisfacción de quien usa el sistema a diario, medida con instrumento	Una calificación media que permita comparar entre versiones	Usuarios reales operando el sistema durante un periodo

Nota. Ninguna de las siete se ha ejecutado a la fecha de este informe. Las cuatro primeras dependen de un entorno de pruebas separado y las tres últimas, de decisiones que exceden al equipo de desarrollo.

La segunda merece una precisión, porque podría parecer que un riesgo conocido quedó sin atender. La prueba de concurrencia sobre la numeración está planificada, pero el riesgo que mediría ya está cubierto por el diseño, con la reserva del consecutivo resuelta en una sola sentencia y un índice único que la base de datos hace cumplir. La prueba serviría para confirmar el comportamiento bajo carga real, no para descubrir si el problema existe.

La última de la tabla es la que más depende de algo ajeno al equipo. La transmisión real exige un proveedor tecnológico habilitado por la autoridad tributaria (DIAN, 2020), y mientras ese acuerdo no exista, lo que se puede verificar es que el sistema produce lo que ese proveedor va a recibir. Eso sí está hecho, con las veinticuatro pruebas del adaptador que fijan el contrato esperado, de modo que una diferencia con el proveedor real aparecerá el primer día de la integración y no en producción.

7. Resultados

La ejecución cerró sin fallas. Las 190 pruebas automatizadas de los 12 archivos terminaron en verde, las 11 peticiones de la interfaz respondieron con el código y la forma documentados, y las 4 operaciones funcionales se completaron con su registro en imagen. No queda ningún defecto de severidad crítica ni mayor abierto.

Ese resultado, por sí solo, dice menos de lo que parece. Una batería que siempre está en verde puede significar que el sistema funciona o que las pruebas no buscan donde duele (Myers et al., 2011), de modo que lo que sigue no es el recuento de lo que pasó sino el de lo que falló mientras se construía, que es donde se ve si el proceso de calidad sirvió para algo.

7.1 Defectos encontrados y corregidos

Durante la construcción se encontraron y corrigieron 4 defectos que vale la pena documentar, no porque sean todos los que hubo, sino porque los cuatro afectaban al documento que el cliente recibe y tres de ellos habrían llegado al usuario final sin que ninguna prueba los detuviera.

Tabla 23

Defectos encontrados durante la construcción y su tratamiento

Defecto	Severidad	Cómo se detectó	Qué lo cubre hoy
El documento salía con el membrete del proveedor y no con el de la empresa que emitía	Crítica	Revisión de un documento generado por la interfaz de integración	Diez pruebas automatizadas, una de las cuales comprueba que el nombre del proveedor no aparezca en ninguna parte del documento de un tercero
El pie mostraba un descuento del 0,0 % restando dinero real	Mayor	Lectura de una factura con descuento por producto	Seis pruebas automatizadas sobre las filas de descuento del pie
La ciudad y el departamento del comprador salían vacíos en todo lo emitido por la interfaz	Mayor	Lectura del documento generado, donde la dirección se leía como si fuera la ciudad	Verificación sobre el documento emitido, pues comprobarlo de forma automatizada exige base de datos

Defecto	Severidad	Cómo se detectó	Qué lo cubre hoy
Las columnas de dinero partían los importes de ocho cifras	Menor	Revisión de una factura de importe alto	El ancho se calcula midiendo el peor caso, y el espacio se toma de la columna que puede repartirse en varias líneas

Nota. Los cuatro están corregidos. La severidad corresponde a la clasificación declarada en el plan de pruebas.

El primero es el más grave que este sistema puede producir y conviene explicar por qué. Los documentos emitidos por la interfaz de integración traen su emisor aparte, y ese dato no llegaba al generador de la representación gráfica, que caía entonces en sus valores por defecto. El resultado era que la factura de una clínica salía con el nombre y el logotipo del proveedor tecnológico, como si la hubiera expedido este. Una factura que dice quién la expidió, nombrando a la empresa equivocada, es un documento que no sirve para lo que existe.

7.2 Lo que los defectos tienen en común

El patrón que forman los cuatro es más útil que cada uno por separado. Ninguno está en la aritmética tributaria, que es la parte del sistema que se escribió con pruebas desde el principio, y los cuatro están en la generación del documento y en la consulta que lo alimenta, que es la parte que no las tenía. Dicho de otro modo, los defectos aparecieron exactamente donde no se estaba mirando.

De ahí salen dos consecuencias que este informe recoge. La primera es la regla que el plan ya declara, según la cual un defecto se reproduce primero con una prueba que falla y solo después se corrige, de modo que hoy dieciséis pruebas cubren dos de esos cuatro defectos y ninguno puede volver en silencio (Beck, 2003). La segunda es la primera acción del plan de mejora, que consiste en extender la automatización a la capa que consulta la base de datos, porque el tercer defecto, la ciudad del comprador que llegaba vacía, sigue sin prueba automatizada justamente por vivir en esa capa.

7.3 Cobertura frente al modelo de calidad adoptado

Cruzando lo ejecutado con las ocho características del capítulo 3, seis quedan respaldadas con evidencia de ejecución y dos lo están solo en parte (ISO/IEC, 2011). La eficiencia de desempeño tiene resueltas por diseño sus tres exigencias, con el correo fuera de la respuesta y los listados paginados, pero le falta la medición del tiempo bajo carga. La fiabilidad tiene probada la atomicidad de la emisión y la independencia frente a la caída del correo, pero su requisito de disponibilidad solo puede verificarse con el sistema desplegado.

Conviene añadir que parte de lo que sostiene la calidad de este sistema no son pruebas sino invariantes, o sea reglas que el diseño impide romper. Están escritas en la documentación técnica del proyecto y son las que explican por qué ciertos errores no pueden ocurrir en lugar de por qué no ocurrieron.

Tabla 24

Invariantes del sistema que el diseño hace cumplir

Invariante	Qué garantiza
Numeración	El consecutivo se reserva en una sola sentencia de actualización, y la tabla tiene un índice único por emisor, tipo y número. Dos documentos con el mismo número de una resolución son dos rechazos
Emisión atómica	Reservar el número, guardar la cabecera, guardar el detalle y mover el inventario van dentro de una transacción, de modo que un fallo a mitad no deja rastro y el consecutivo queda libre
Cálculo	La aritmética tributaria vive en un solo módulo, que es el que alimenta por igual al formulario web y a la API, y es el que está cubierto por pruebas
Inventario	Toda modificación de existencias pasa por su servicio y deja su fila en el kardex, de modo que el saldo siempre es reconstruible
Membrete	El documento lleva siempre el emisor que lo expide, nunca el del proveedor tecnológico
Auditoría	El registro solo se escribe, nunca se corrige, guarda el nombre del usuario y no puede tumbar la operación que está anotando

Nota. Un invariante es más fuerte que una prueba, porque la prueba comprueba un caso y el invariante impide todos. La reserva del consecutivo es el ejemplo claro, ya que el índice único de la base rechaza el segundo documento aunque el programa se equivoque.

8. Conclusiones y plan de mejora

El trabajo de calidad sobre FactuGest permite sostener cinco conclusiones.

1. La norma ISO/IEC 25010 resultó aplicable al proyecto sin forzarla, y el cruce con el catálogo propio dejó las ocho características cubiertas. El reparto resultante, con la seguridad concentrando la mayor parte de los requisitos, describe con precisión lo que significa operar como proveedor tecnológico, y no es un resultado que se hubiera podido anticipar sin hacer el cruce.
2. Adoptar un referente externo obligó a renombrar tres categorías propias que parecían correctas. La trazabilidad resultó ser responsabilidad y no repudio, y la escalabilidad se repartió entre capacidad e interoperabilidad, lo que muestra que el vocabulario común de un estándar no es un formalismo sino una forma de clasificar mejor lo que ya se tenía.
3. La automatización cumplió su función donde se aplicó. Las pruebas de la aritmética tributaria, escritas con valores calculados a mano, no dejaron pasar ningún defecto de cálculo, que es la clase de error más costosa en un documento con efectos tributarios.
4. Los cuatro defectos documentados aparecieron en la parte del sistema que no tenía pruebas, lo que confirma que la cobertura no es un indicador decorativo. El más grave de todos, el membrete con el emisor equivocado, no habría producido un error visible en ninguna pantalla, y por eso solo aparece cuando alguien revisa el documento resultante.
5. El aseguramiento de la calidad no se sostiene únicamente en pruebas. Los invariantes de numeración, atomicidad y cálculo único impiden clases enteras de error que una prueba solo podría detectar caso por caso, y son la respuesta correcta para lo que depende de la concurrencia.

De las pruebas planificadas y de lo aprendido en las ejecutadas sale el plan de mejora, ordenado por lo que más reduce el riesgo y no por lo que cuesta menos.

Tabla 25

Plan de mejora continua

Acción	Qué riesgo corrige	Cuándo
Automatizar la ejecución de la batería con cada cambio integrado	Que una corrección rompa algo probado y nadie se entere hasta la siguiente corrida	Junto con el despliegue
Extender las pruebas automatizadas a la capa que consulta la base de datos	Que los defectos aparezcan justamente donde no hay pruebas, que es lo que ocurrió con los cuatro encontrados	Antes del despliegue
Medir el tiempo de respuesta de la emisión bajo carga	Que el primer requisito no funcional siga siendo una intención y no una medida	Junto con el despliegue
Incorporar el análisis de dependencias a la rutina	Que una biblioteca con una vulnerabilidad conocida entre sin que nadie lo note	Antes del despliegue
Formalizar el acuerdo con un proveedor tecnológico habilitado	Que la transmisión real ante la autoridad tributaria siga fuera del alcance	Fase siguiente del proyecto
Aplicar el instrumento de usabilidad después de un periodo de uso	Que la usabilidad se sostenga en decisiones de diseño y no en una medida	Con el sistema en operación

Nota. Las dos primeras acciones salen de los defectos encontrados y no de una lista general de buenas prácticas.

Queda por último una observación sobre el alcance de todo lo anterior (ISTQB, 2018).

Ninguna de estas pruebas demuestra que FactuGest esté libre de defectos, porque eso no es algo que las pruebas puedan demostrar. Lo que sostienen las 190 pruebas automatizadas, las peticiones sobre la interfaz, las operaciones verificadas en pantalla y los invariantes del diseño es algo más modesto y más útil, y es que los errores que ya se cometieron una vez no pueden volver sin que alguien se entere.

9. Referencias

Las referencias se presentan en orden alfabético, según el estilo APA en su séptima edición. Las normas se citan por el organismo que las publica y el año de la versión utilizada.

Alfonzo, P., y Mariño, S. (2013). Propuesta metodológica para la evaluación de la calidad de productos de software. Universidad Nacional del Nordeste.

Beck, K. (2003). Test-driven development: by example. Addison-Wesley.

CMMI Product Team. (2010). CMMI for Development, Version 1.3 (CMU/SEI-2010-TR-033). Software Engineering Institute, Carnegie Mellon University.

Dirección de Impuestos y Aduanas Nacionales. (2020). Resolución 000042 de 2020, por la cual se desarrollan los sistemas de facturación, los proveedores tecnológicos y se expide el anexo técnico de factura electrónica de venta. DIAN.

Estayno, M., Dapozo, G., Cuenca Pletsch, L., y Greiner, C. (2009). Modelos y métricas para evaluar la calidad del software. XI Workshop de Investigadores en Ciencias de la Computación, 382-386.

Fillottrani, P. (2007). Calidad en el desarrollo de software: modelos de calidad de software. Universidad Nacional del Sur.

Gómez, D. (2009). ¿Qué significa calidad de software? Dos Ideas.

Institute of Electrical and Electronics Engineers. (1990). IEEE Standard Glossary of Software Engineering Terminology (IEEE Std 610.12-1990). IEEE.

Institute of Electrical and Electronics Engineers. (2008). IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008). IEEE.

- International Organization for Standardization. (2001). ISO/IEC 9126-1:2001. Software engineering. Product quality. Part 1, Quality model. ISO.
- International Organization for Standardization. (2004). ISO/IEC 15504-1:2004. Information technology. Process assessment. Part 1, Concepts and vocabulary. ISO.
- International Organization for Standardization. (2005). ISO/IEC 25000:2005. Software engineering. Software product Quality Requirements and Evaluation (SQuaRE). Guide to SQuaRE. ISO.
- International Organization for Standardization. (2008). ISO/IEC 12207:2008. Systems and software engineering. Software life cycle processes. ISO.
- International Organization for Standardization. (2011). ISO/IEC 25010:2011. Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models. ISO.
- International Organization for Standardization. (2013). ISO/IEC/IEEE 29119-1:2013. Software and systems engineering. Software testing. Part 1, Concepts and definitions. ISO.
- International Organization for Standardization. (2015). ISO 9001:2015. Quality management systems. Requirements. ISO.
- International Software Testing Qualifications Board. (2018). Certified Tester Foundation Level Syllabus, Version 2018. ISTQB.
- Myers, G. J., Sandler, C., y Badgett, T. (2011). The art of software testing (3.^a ed.). John Wiley and Sons.
- Polillo, R. (2011). Quality models for web [2.0] sites: a methodological approach and a proposal. Lecture Notes in Computer Science, 6757, 251-265.
- Portal ISO 25000. (2018). La familia de normas ISO/IEC 25000.

Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7.^a ed.). McGraw-Hill.

Real Academia Española. (2023). Diccionario de la lengua española (23.^a ed.).

Scalone, F. (2006). Estudio comparativo de los modelos y estándares de calidad del software [Tesis de maestría, Universidad Tecnológica Nacional].

Servicio Nacional de Aprendizaje. (s.f.). Modelos y estándares de calidad del software [Material de formación]. Formación en Ambientes Virtuales de Aprendizaje, SENA.

Vega, V., Rivera, L., y García, R. (2008). Mejores prácticas para el establecimiento y aseguramiento de la calidad de software. Universidad Cristóbal Colón.

10. Anexos

Se relacionan las evidencias que respaldan lo afirmado en el informe. Las tres primeras se producen solas al ejecutar el sistema o su batería de pruebas, de modo que cualquiera puede reproducirlas.

Tabla 26

Anexos del informe

Anexo	Contenido	Dónde está
A	Salida de la corrida de la batería de pruebas	Se transcribe a continuación y se conserva en el repositorio del proyecto
B	Listado de las 190 pruebas superadas, con su nombre	Se transcribe a continuación, obtenido de la misma corrida
C	Colección de peticiones de prueba de la interfaz de integración	En el repositorio del proyecto, en formato de intercambio de la herramienta
D	Imágenes de las pruebas funcionales	En el repositorio del proyecto, ocho imágenes de cuatro operaciones
E	Catálogo de requisitos funcionales y no funcionales	Entregable independiente del proyecto, del que salen los requisitos clasificados en el capítulo 3

Nota. Los anexos A y B se generan al ejecutar la batería y no se escriben a mano, de manera que corresponden siempre a la última corrida.

Anexo A. Salida de la corrida de pruebas

Corrida del 8 de septiembre de 2026 con pytest 9.1.1 sobre Python 3.13.3. La batería completa terminó en 3,39 segundos sin ninguna falla.

Tabla 27

Resultado de la corrida por archivo de pruebas

Archivo	Pruebas	Resultado
test_validaciones.py	45	Todas superadas
test_dian_proveedor.py	24	Todas superadas
test_modelos_api.py	22	Todas superadas
test_calculo_documento.py	19	Todas superadas
test_calculo_notas.py	16	Todas superadas
test_monograma.py	13	Todas superadas
test_api_key_service.py	11	Todas superadas
test_pdf_membrete.py	10	Todas superadas
test_consumo_service.py	9	Todas superadas
test_xml_service.py	9	Todas superadas
test_documento_canonico.py	6	Todas superadas
test_pdf_descuentos.py	6	Todas superadas
Total	190	Sin fallas

Nota. La última fila es el resultado de la sesión completa, tal como lo reporta la herramienta al terminar.

Anexo B. Listado de las pruebas superadas

El nombre de cada prueba se deja tal como está escrito en el código, porque está redactado para decir qué comprueba y no para nombrar una función. Leído de corrido, este listado es la descripción más precisa de lo que el sistema garantiza. Entre corchetes aparecen los datos con los que se ejecutó una prueba escrita una vez y corrida con varios juegos de valores.

Tabla 28*Las 190 pruebas superadas en la corrida*

N.º	Archivo	Prueba
1	test_api_key_service.py	La llave se parte en prefijo y secreto
2	test_api_key_service.py	El prefijo cabe en la columna y es reconocible
3	test_api_key_service.py	El secreto tiene entropia suficiente
4	test_api_key_service.py	Dos llaves seguidas no se parecen
5	test_api_key_service.py	Una llave mal formada se rechaza antes de consultar nada []
6	test_api_key_service.py	Una llave mal formada se rechaza antes de consultar nada [None]
7	test_api_key_service.py	Una llave mal formada se rechaza antes de consultar nada [sinpunto]
8	test_api_key_service.py	Una llave mal formada se rechaza antes de consultar nada [solosecreto]
9	test_api_key_service.py	Una llave mal formada se rechaza antes de consultar nada [soloprefijo.]
10	test_api_key_service.py	Una llave mal formada se rechaza antes de consultar nada []
11	test_api_key_service.py	El hash es estable y no deja ver el secreto
12	test_calculo_documento.py	Linea simple con iva
13	test_calculo_documento.py	Linea con descuento propio
14	test_calculo_documento.py	Linea exenta no genera impuesto
15	test_calculo_documento.py	La linea conserva las claves ajenas
16	test_calculo_documento.py	Documento de una linea
17	test_calculo_documento.py	Tarifas mixtas en el mismo documento
18	test_calculo_documento.py	Descuento global prorratea el iva
19	test_calculo_documento.py	Descuento global con tarifas mixtas
20	test_calculo_documento.py	Descuento global sobre documento sin base no divide por cero
21	test_calculo_documento.py	Valores ausentes o vacios se tratan como cero
22	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas0-0]
23	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas1-10000]
24	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas2-0]
25	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas3-7777]
26	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas4-0]
27	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas5-3333]
28	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas6-0]
29	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas7-9000]
30	test_calculo_documento.py	Los totales coinciden con el calculo anterior [lineas8-12345]
31	test_calculo_notas.py	La anulacion total devuelve exactamente lo facturado
32	test_calculo_notas.py	Los importes de la nota son positivos
33	test_calculo_notas.py	La devolucion parcial prorratea base descuento e impuesto
34	test_calculo_notas.py	Devolver sin prorratar el descuento regresaria de mas

N.º	Archivo	Prueba
35	test_calculo_notas.py	Una linea no devuelta no entra en la nota
36	test_calculo_notas.py	Devolver cero deja la nota vacia
37	test_calculo_notas.py	La nota conserva el orden de la linea original
38	test_calculo_notas.py	El valor con iva adentro se descompone
39	test_calculo_notas.py	El valor sin iva lo suma encima
40	test_calculo_notas.py	Las dos formas no dan lo mismo
41	test_calculo_notas.py	Un ajuste sin impuesto no inventa iva
42	test_calculo_notas.py	La tarifa promedio de un documento de una sola tarifa es esa tarifa
43	test_calculo_notas.py	La tarifa promedio de un documento mixto queda en medio
44	test_calculo_notas.py	Un documento sin base no divide por cero [0]
45	test_calculo_notas.py	Un documento sin base no divide por cero [None]
46	test_calculo_notas.py	Un documento sin base no divide por cero [-5]
47	test_consumo_service.py	Un plan sin tope nunca genera excedente
48	test_consumo_service.py	Por debajo del cupo no se cobra nada de mas
49	test_consumo_service.py	El excedente es lo que pasa del cupo y no el total
50	test_consumo_service.py	Justo en el cupo todavia no hay excedente
51	test_consumo_service.py	El semaforo avisa antes de que le rebote una emision [319-NORMAL]
52	test_consumo_service.py	El semaforo avisa antes de que le rebote una emision [320-ALERTA]
53	test_consumo_service.py	El semaforo avisa antes de que le rebote una emision [401-EXCEDIDO]
54	test_consumo_service.py	Un cliente que no emitio nada no rompe el calculo
55	test_consumo_service.py	El conteo llega como decimal desde mysql
56	test_dian_proveedor.py	Un documento coherente se acepta
57	test_dian_proveedor.py	La respuesta trae el qr del catalogo de habilitacion
58	test_dian_proveedor.py	El acuse es reproducible para el mismo documento
59	test_dian_proveedor.py	El payload avisa que no hubo transmision real
60	test_dian_proveedor.py	El simulador no inventa numero ni cufe ni pdf
61	test_dian_proveedor.py	Se rechaza lo que la dian rechazaria [cambio0-no tiene n\xfamero]
62	test_dian_proveedor.py	Se rechaza lo que la dian rechazaria [cambio1-no tiene CUFE]
63	test_dian_proveedor.py	Se rechaza lo que la dian rechazaria [cambio2-forma de un SHA-384]
64	test_dian_proveedor.py	Se rechaza lo que la dian rechazaria [cambio3-forma de un SHA-384]
65	test_dian_proveedor.py	Se rechaza lo que la dian rechazaria [cambio4-receptor no est\xe1 identificado]
66	test_dian_proveedor.py	Se rechaza un emisor sin nit
67	test_dian_proveedor.py	Se rechaza un documento sin lineas
68	test_dian_proveedor.py	Se rechaza un total que no cuadra
69	test_dian_proveedor.py	Se rechaza un iva mayor que el de las lineas
70	test_dian_proveedor.py	Se rechaza un xml mal formado
71	test_dian_proveedor.py	Se rechaza si no se genero el xml
72	test_dian_proveedor.py	Se informan todos los problemas de una vez
73	test_dian_proveedor.py	Un rechazo no lanza excepcion

N.º	Archivo	Prueba
74	test_dian_proveedor.py	Una nota credito con valores negativos se acepta
75	test_dian_proveedor.py	Por defecto se usa el simulado
76	test_dian_proveedor.py	La variable de entorno elige el proveedor
77	test_dian_proveedor.py	Factus avisa que esta pendiente en lugar de decir desconocido
78	test_dian_proveedor.py	Un proveedor inventado lista los disponibles
79	test_dian_proveedor.py	Los disponibles se pueden consultar
80	test_documento_canonico.py	Los alias de la consulta se traducen al nombre de columna
81	test_documento_canonico.py	El emisor extraido trae el contrato completo
82	test_documento_canonico.py	El emisor desde empresa rellena lo que no selecciono la consulta
83	test_documento_canonico.py	No hay claves faltantes cuando el documento esta completo
84	test_documento_canonico.py	Se reportan las claves ausentes de cada pieza
85	test_documento_canonico.py	Una clave presente en nulo cuenta como presente
86	test_modelos_api.py	Una factura minima es valida
87	test_modelos_api.py	Los datos se normalizan al validar
88	test_modelos_api.py	La cedula no admite letras
89	test_modelos_api.py	El pasaporte si admite letras
90	test_modelos_api.py	El tipo de documento es del catalogo dian
91	test_modelos_api.py	Una persona natural no lleva numeros en el nombre
92	test_modelos_api.py	Una persona juridica si
93	test_modelos_api.py	El regimen y el tipo de persona son listas cerradas
94	test_modelos_api.py	El correo debe tener forma de correo
95	test_modelos_api.py	El precio debe ser mayor que cero
96	test_modelos_api.py	La cantidad admite decimales pero no cero
97	test_modelos_api.py	El descuento de linea va de cero a cien
98	test_modelos_api.py	El codigo de impuesto es del anexo dian
99	test_modelos_api.py	El codigo del item no se valida contra nuestro catalogo
100	test_modelos_api.py	Una factura necesita al menos una linea
101	test_modelos_api.py	A credito hace falta plazo
102	test_modelos_api.py	De contado no lleva plazo
103	test_modelos_api.py	El plazo tiene un tope razonable
104	test_modelos_api.py	El descuento global no puede pasarse de la base
105	test_modelos_api.py	El descuento global no puede ser negativo
106	test_modelos_api.py	El descuento global considera los descuentos de linea
107	test_modelos_api.py	El ejemplo que muestra swagger se puede enviar
108	test_monograma.py	Toma las dos primeras palabras con significado [Drogueria La Salud S.A.S.-DS]
109	test_monograma.py	Toma las dos primeras palabras con significado [Transportes del Norte S.A.S.-TN]
110	test_monograma.py	Toma las dos primeras palabras con significado [Hotel Casa Blanca-HC]
111	test_monograma.py	Toma las dos primeras palabras con significado [Ferreteria Los Andes Ltda-FA]
112	test_monograma.py	Toma las dos primeras palabras con significado [Droguer\xeda La Salud-DS]

N.º	Archivo	Prueba
113	test_monograma.py	Un nombre de una sola palabra usa sus dos mayusculas
114	test_monograma.py	Un nombre sin mayusculas usa sus dos primeras letras
115	test_monograma.py	Un nombre vacio no revienta
116	test_monograma.py	El color de la empresa manda sobre el derivado
117	test_monograma.py	Sin color propio el derivado es siempre el mismo
118	test_monograma.py	Un color mal escrito no se cuela en el documento
119	test_monograma.py	Dos empresas distintas no tienen que verse iguales
120	test_monograma.py	El pdf dibuja el monograma cuando no hay logo
121	test_pdf_descuentos.py	El descuento por producto sale con su porcentaje real
122	test_pdf_descuentos.py	El descuento de factura sale con su concepto
123	test_pdf_descuentos.py	Los dos descuentos salen en filas distintas
124	test_pdf_descuentos.py	El de factura se calcula sobre lo que queda del de linea
125	test_pdf_descuentos.py	Una factura sin descuentos no muestra la fila
126	test_pdf_descuentos.py	El concepto del descuento de linea sigue bajo el producto
127	test_pdf_membrete.py	El membrete es del emisor que se le pasa
128	test_pdf_membrete.py	El membrete nunca cae en el nuestro
129	test_pdf_membrete.py	Sin emisor explicito lo lee del documento
130	test_pdf_membrete.py	Sin logo el membrete imprime el nombre una sola vez
131	test_pdf_membrete.py	Un logo que no existe no tumba la factura
132	test_pdf_membrete.py	Un logo con ruta de escape se ignora
133	test_pdf_membrete.py	Cada emisor pinta con su color
134	test_pdf_membrete.py	Un color mal escrito no tumba la factura
135	test_pdf_membrete.py	Sin color sale el neutro y no el de nadie
136	test_pdf_membrete.py	El pdf no lleva ningun color de marca escrito en el codigo
137	test_validaciones.py	Entero acepta cero y positivos
138	test_validaciones.py	Entero rechaza negativos
139	test_validaciones.py	Entero rechaza decimales y texto
140	test_validaciones.py	Entero respeta el maximo
141	test_validaciones.py	Los numeros admiten coma decimal y espacios
142	test_validaciones.py	El dinero no puede ser negativo pero si cero
143	test_validaciones.py	El precio debe ser mayor que cero
144	test_validaciones.py	La cantidad arranca en uno
145	test_validaciones.py	La cantidad fraccionaria es opcional
146	test_validaciones.py	El porcentaje va de cero a cien
147	test_validaciones.py	Infinito y nan no pasan por numero
148	test_validaciones.py	El nombre de persona admite tildes y apostrofes
149	test_validaciones.py	El nombre de persona rechaza numeros y simbolos
150	test_validaciones.py	La razon social si admite numeros
151	test_validaciones.py	La razon social necesita al menos una letra

N.º	Archivo	Prueba
152	test_validaciones.py	El texto respeta el largo
153	test_validaciones.py	El sku se normaliza en mayusculas
154	test_validaciones.py	El correo valida forma y se normaliza
155	test_validaciones.py	El telefono cuenta digitos
156	test_validaciones.py	La cedula solo admite numeros
157	test_validaciones.py	El pasaporte admite letras
158	test_validaciones.py	La identificacion ignora puntos y guiones de separacion
159	test_validaciones.py	La identificacion respeta el largo de su tipo
160	test_validaciones.py	Las etiquetas traducen el codigo para las personas
161	test_validaciones.py	El tipo de documento debe ser del catalogo dian
162	test_validaciones.py	El digito de verificacion sigue el algoritmo dian [890903938-8]
163	test_validaciones.py	El digito de verificacion sigue el algoritmo dian [800197268-4]
164	test_validaciones.py	El digito de verificacion sigue el algoritmo dian [900373115-3]
165	test_validaciones.py	El dv calculado siempre se verifica contra su propio nit
166	test_validaciones.py	El dv siempre queda entre cero y nueve
167	test_validaciones.py	El dv se calcula si viene vacio y se verifica si viene
168	test_validaciones.py	El rango de numeracion exige desde menor o igual que hasta
169	test_validaciones.py	El rango no admite negativos ni cero
170	test_validaciones.py	El rango puede quedar vacio si no es obligatorio
171	test_validaciones.py	Las fechas de vigencia deben ir en orden
172	test_validaciones.py	Una fecha mal escrita se rechaza
173	test_validaciones.py	El nit solo admite digitos y de 8 a 10
174	test_validaciones.py	El ciiu son cuatro digitos
175	test_validaciones.py	El prefijo se normaliza y no admite simbolos
176	test_validaciones.py	La contrasena exige largo minimo salvo al editar
177	test_validaciones.py	El sitio web rechaza lo que no parece dominio
178	test_validaciones.py	La pareja asigna los dos campos o marca el que falla
179	test_validaciones.py	La opcion debe estar en la lista blanca
180	test_validaciones.py	El validador acumula todos los errores del formulario
181	test_validaciones.py	El validador queda valido cuando todo pasa
182	test_xml_service.py	El regimen del emisor define el nivel tributario [RESPONSABLE_IVA-O-23]
183	test_xml_service.py	El regimen del emisor define el nivel tributario [GRAN_CONTRIBUYENTE-O-23]
184	test_xml_service.py	El regimen del emisor define el nivel tributario [NO_RESPONSABLE_IVA-O-47]
185	test_xml_service.py	Sin regimen se asume responsable de iva
186	test_xml_service.py	El tipo de documento del cliente viaja como codigo dian [13-13]
187	test_xml_service.py	El tipo de documento del cliente viaja como codigo dian [31-31]
188	test_xml_service.py	El tipo de documento del cliente viaja como codigo dian [41-41]
189	test_xml_service.py	El tipo de documento del cliente viaja como codigo dian [ZZ-13]
190	test_xml_service.py	El prefijo del emisor llega al xml

Nota. Listado obtenido de la salida de la corrida, sin edición manual.